

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web 2

Entrega 4B

Panel Admin — Gestión de Usuarios

FASE 2 — Gestión de Usuarios y Materias

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear API Routes protegidas exclusivas para el rol administrador
- Implementar un endpoint GET para listar todos los usuarios del sistema
- Implementar un endpoint PUT para activar, desactivar y cambiar el rol de usuarios
- Construir una tabla de datos con el componente Table de shadcn/ui
- Agregar filtros interactivos por rol y estado usando el componente Select
- Usar componentes Badge para indicadores visuales de estado y rol
- Implementar diálogos de confirmación con AlertDialog antes de acciones destructivas
- Aplicar el patrón Server Component (datos) + Client Component (interactividad)

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entrega 1A — Setup del Proyecto
- ✓ Entrega 1B — shadcn/ui y Base de Datos
- ✓ Entrega 2A — Clientes Supabase y Proxy
- ✓ Entrega 2B — Login, Registro y Redirect por Rol
- ✓ Entrega 3A — Layouts y Sidebar
- ✓ Entrega 3B — Header, Logout y Perfil
- ✓ Entrega 4A — Página de Perfil
- ✓ Tener al menos 3 usuarios de prueba creados (1 admin, 1 docente, 1 estudiante)

i Importante: Necesitarás al menos un usuario con rol "admin" para acceder al panel. Si aún no tienes uno, cambia el rol directamente en Supabase Dashboard: Table Editor → usuarios_perfil → editar campo rol.

Teoría: Paneles de Administración y Control de Acceso

En toda aplicación web con múltiples roles, se necesita un panel de administración donde un usuario privilegiado pueda gestionar a los demás. Esto implica dos conceptos clave:

Control de Acceso Basado en Roles (RBAC)

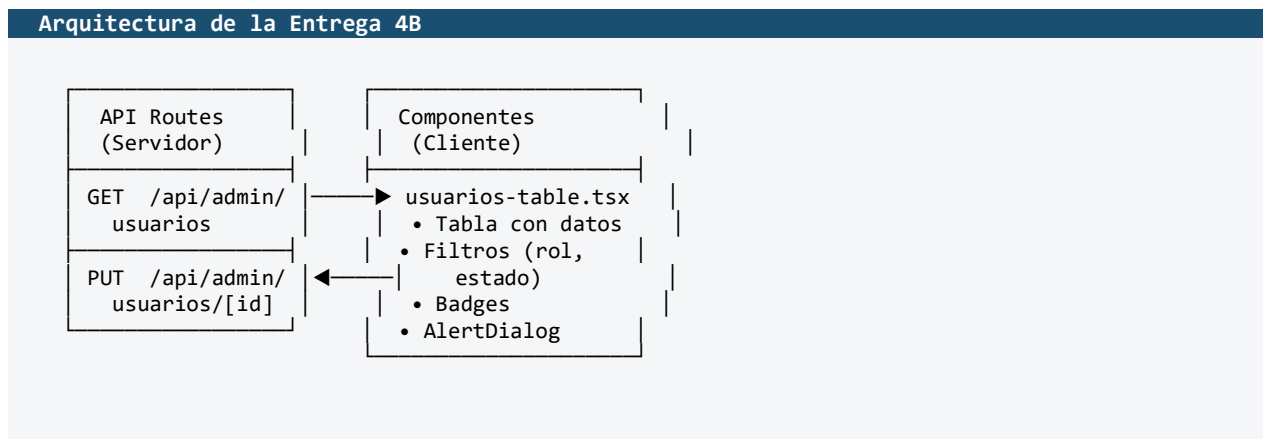
RBAC (Role-Based Access Control) es el modelo de seguridad que usamos en gestion-academica. Cada usuario tiene un rol (estudiante, docente, admin) que determina qué puede hacer:

Capa de protección	Dónde se aplica	Qué protege
proxy.ts	Servidor (cada request)	Redirige usuarios no autenticados o sin rol correcto
API Route	Servidor (endpoint)	Verifica rol antes de ejecutar la operación
Interfaz (UI)	Cliente (navegador)	Oculto opciones según el rol del usuario

⚠ Seguridad: Nunca confíes solo en la UI para proteger funcionalidades. Un usuario puede llamar directamente a tu API con herramientas como curl o Postman. Siempre valida el rol en el servidor.

Patrón de la Entrega

En esta entrega seguiremos el flujo de datos que ya conocemos:



Service Role Key vs Publishable Key

En las API Routes de admin usaremos la SERVICE_ROLE_KEY para poder leer y modificar usuarios sin restricciones de RLS. Recordemos la diferencia:

Clave	Uso	Permisos
PUBLISHABLE_KEY	Cliente (navegador)	Respetar RLS, solo lo que el usuario puede ver
SERVICE_ROLE_KEY	Servidor (API Routes)	Ignora RLS, acceso total a todas las tablas

✗ Cuidado: NUNCA expongas la SERVICE_ROLE_KEY en el cliente. Solo úsala en archivos dentro de app/api/ que se ejecutan exclusivamente en el servidor.

Paso 1 — Instalar Componentes de shadcn/ui

Necesitamos 4 componentes nuevos para esta entrega. Abre la terminal en la raíz de tu proyecto y ejecuta:

Terminal

```
npx shadcn@latest add table
npx shadcn@latest add badge
npx shadcn@latest add alert-dialog
npx shadcn@latest add select
```

Verifiquemos que se crearon los archivos:

Componente	Archivo generado	Para qué lo usaremos
Table	components/ui/table.tsx	Mostrar la lista de usuarios en formato tabla
Badge	components/ui/badge.tsx	Indicadores visuales de rol y estado
AlertDialog	components/ui/alert-dialog.tsx	Confirmación antes de cambios críticos
Select	components/ui/select.tsx	Filtros desplegables de rol y estado

☒ **Verificación:** Abre la carpeta components/ui/ en VS Code y confirma que los 4 archivos nuevos existen. Si alguno falta, vuelve a ejecutar el comando correspondiente.

Paso 2 — API Route: Listar Usuarios (GET)

Crearemos un endpoint que devuelve todos los usuarios del sistema. Solo los administradores pueden acceder a este endpoint.

2.1 — Crear la estructura de carpetas

Terminal

```
mkdir -p app/api/admin/usuarios
```

2.2 — Crear el archivo de la API Route

Crema el archivo `app/api/admin/usuarios/route.ts` con el siguiente contenido:

app/api/admin/usuarios/route.ts

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    // 1. Verificar autenticación del usuario actual
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar que el usuario es admin
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 3. Usar Service Role para listar TODOS los usuarios
    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    const { data: usuarios, error } = await supabaseAdmin
```

```

    .from("usuarios_perfil")
    .select("id, nombre, apellido, rol, activo, created_at, updated_at")
    .order("created_at", { ascending: false });

    if (error) {
      console.error("Error al listar usuarios:", error);
      return NextResponse.json({ error: "Error al obtener usuarios" }, { status: 500 });
    }

    // 4. Obtener emails de auth.users
    const { data: authData } = await supabaseAdmin.auth.admin.listUsers();

    // 5. Combinar perfil + email
    const usuariosConEmail = usuarios?.map((u) => {
      const authUser = authData?.users?.find((au) => au.id === u.id);
      return {
        ...u,
        email: authUser?.email || "Sin email",
      };
    });

    return NextResponse.json({ usuarios: usuariosConEmail });
  } catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json(
      { error: "Error interno del servidor" },
      { status: 500 }
    );
  }
}

```

2.3 — Explicación del código

Analicemos las secciones clave de esta API Route:

Sección	Qué hace	Por qué es importante
Verificar autenticación	Comprueba que hay un usuario logueado	Sin esto, cualquier persona podría acceder
Verificar rol admin	Consulta el perfil y verifica rol === admin	Segunda capa: solo admins pueden listar
Service Role client	Crea un cliente con SERVICE_ROLE_KEY	Ignora RLS para ver todos los registros
Combinar datos	Une perfil (tabla) + email (auth)	El email está en auth.users, no en nuestra tabla

[i] Nota: Usamos `auth.admin.listUsers()` para obtener los emails. Esta función solo funciona con la `SERVICE_ROLE_KEY` porque accede a la tabla interna `auth.users` de Supabase.

Paso 3 — API Route: Modificar Usuario (PUT)

Este endpoint permitirá al admin cambiar el rol de un usuario o activar/desactivar su cuenta.

3.1 — Crear la estructura para rutas dinámicas

Terminal

```
mkdir -p "app/api/admin/usuarios/[id]"
```

i Nota: Los corchetes [id] en el nombre de la carpeta indican una ruta dinámica en Next.js. El valor de id se recibirá como parámetro.

3.2 — Crear el archivo de la API Route

Crea el archivo `app/api/admin/usuarios/[id]/route.ts`:

app/api/admin/usuarios/[id]/route.ts

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextRequest, NextResponse } from "next/server";

export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params;

    // 1. Verificar autenticación
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar rol admin
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }
  }
}
```

```

}

// 3. Evitar que el admin se modifique a sí mismo
if (id === user.id) {
  return NextResponse.json(
    { error: "No puedes modificar tu propia cuenta" },
    { status: 400 }
  );
}

// 4. Leer los datos del body
const body = await request.json();
const { rol, activo } = body;

// 5. Validar los campos
const rolesValidos = ["estudiante", "docente", "admin"];
if (rol !== undefined && !rolesValidos.includes(rol)) {
  return NextResponse.json({ error: "Rol inválido" }, { status: 400 });
}

if (activo !== undefined && typeof activo !== "boolean") {
  return NextResponse.json(
    { error: "El campo activo debe ser true o false" },
    { status: 400 }
  );
}

// 6. Construir objeto de actualización
const actualizacion: Record<string, unknown> = {};
if (rol !== undefined) actualizacion.rol = rol;
if (activo !== undefined) actualizacion.activo = activo;

if (Object.keys(actualizacion).length === 0) {
  return NextResponse.json(
    { error: "No se proporcionaron campos para actualizar" },
    { status: 400 }
  );
}

// 7. Actualizar con Service Role
const supabaseAdmin = createServiceClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

const { data, error } = await supabaseAdmin
  .from("usuarios_perfil")
  .update(actualizacion)
  .eq("id", id)
  .select("id, nombre, apellido, rol, activo")
  .single();

if (error) {
  console.error("Error al actualizar usuario:", error);
  return NextResponse.json(
    { error: "Error al actualizar el usuario" },
    { status: 500 }
  );
}

```

```

    return NextResponse.json({ usuario: data });
  } catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json(
      { error: "Error interno del servidor" },
      { status: 500 }
    );
  }
}

```

3.3 — Puntos clave del código

Concepto	Detalle
params es Promise	En Next.js 16, params es async. Usamos await params para obtener { id }
Auto-modificación bloqueada	El admin no puede cambiar su propio rol ni desactivarse
Validación manual	Verificamos que rol sea válido y activo sea booleano
Actualización parcial	Solo enviamos los campos que realmente cambiaron
Service Role	Usamos SERVICE_ROLE_KEY para bypassar RLS en la actualización

☒ **Buena práctica:** Siempre bloquea la auto-modificación del admin. Si un admin se desactiva a sí mismo, nadie podrá volver a activar cuentas.

Paso 4 — Componente: Tabla de Usuarios

Este es el componente principal de la entrega. Es un Client Component que muestra una tabla interactiva con filtros, badges y diálogos de confirmación.

4.1 — Crear la carpeta del componente

Terminal

```
mkdir -p components/dashboard/admin
```

4.2 — Crear el componente

Crea el archivo `components/dashboard/admin/usuarios-table.tsx`. Este es un archivo extenso, así que lo iremos explicando por secciones:

Sección A — Imports y tipos

components/dashboard/admin/usuarios-table.tsx — Parte 1/4

```
"use client";

import { useEffect, useState } from "react";
import { createClient } from "@lib/supabase/client";
import { toast } from "sonner";

// Componentes shadcn/ui
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader,
  AlertDialogTitle, AlertDialogTrigger,
} from "@components/ui/alert-dialog";

// Iconos
import { Shield, UserCheck, UserX, RefreshCw } from "lucide-react";

// Tipo para los usuarios
interface Usuario {
  id: string;
  nombre: string;
```

```
apellido: string;  
email: string;  
rol: "estudiante" | "docente" | "admin";  
activo: boolean;  
created_at: string;  
}
```

i Nota: Definimos la interfaz Usuario directamente en el componente. En proyectos grandes, estos tipos irían en un archivo separado como lib/types.ts.

Sección B — Estado y carga de datos

```
components/dashboard/admin/usuarios-table.tsx – Parte 2/4
export default function UsuariosTable() {
  // Estados
  const [usuarios, setUsuarios] = useState<Usuario[]>([]);
  const [loading, setLoading] = useState(true);
  const [filtroRol, setFiltroRol] = useState<string>("todos");
  const [filtroEstado, setFiltroEstado] = useState<string>("todos");

  // Función para cargar usuarios
  const cargarUsuarios = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/admin/usuarios");
      const data = await res.json();

      if (!res.ok) {
        toast.error(data.error || "Error al cargar usuarios");
        return;
      }

      setUsuarios(data.usuarios);
    } catch {
      toast.error("Error de conexión");
    } finally {
      setLoading(false);
    }
  };

  // Cargar al montar el componente
  useEffect(() => {
    cargarUsuarios();
  }, []);

  // Filtrar usuarios
  const usuariosFiltrados = usuarios.filter((u) => {
    const cumpleRol = filtroRol === "todos" || u.rol === filtroRol;
    const cumpleEstado =
      filtroEstado === "todos" ||
      (filtroEstado === "activo" && u.activo) ||
      (filtroEstado === "inactivo" && !u.activo);
    return cumpleRol && cumpleEstado;
  });
}
```

☑ **Patrón:** Separamos `cargarUsuarios` como función independiente para poder reutilizarla después de cada actualización (refresco de tabla).

Sección C — Funciones de acción

components/dashboard/admin/usuarios-table.tsx — Parte 3/4

```
// Cambiar rol de un usuario
const cambiarRol = async (id: string, nuevoRol: string) => {
  try {
    const res = await fetch(`/api/admin/usuarios/${id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ rol: nuevoRol }),
    });

    const data = await res.json();

    if (!res.ok) {
      toast.error(data.error || "Error al cambiar rol");
      return;
    }

    toast.success(`Rol actualizado a "${nuevoRol}"`);
    cargarUsuarios(); // Refrescar tabla
  } catch {
    toast.error("Error de conexión");
  }
};

// Activar o desactivar usuario
const toggleActivo = async (id: string, estadoActual: boolean) => {
  try {
    const res = await fetch(`/api/admin/usuarios/${id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ activo: !estadoActual }),
    });

    const data = await res.json();

    if (!res.ok) {
      toast.error(data.error || "Error al cambiar estado");
      return;
    }

    toast.success(
      !estadoActual ? "Usuario activado" : "Usuario desactivado"
    );
    cargarUsuarios();
  } catch {
    toast.error("Error de conexión");
  }
};
```

Observa cómo ambas funciones siguen el mismo patrón:

- Hacer fetch al endpoint PUT con el campo específico a cambiar
- Mostrar toast de éxito o error

- Llamar `cargarUsuarios()` para refrescar la tabla con datos actualizados

Sección D — Renderizado (JSX)

La sección de renderizado incluye los filtros, la tabla y los diálogos. Continuamos en el mismo archivo:

```
components/dashboard/admin/usuarios-table.tsx — Parte 4/4
// Helper para color del badge de rol
const badgeRol = (rol: string) => {
  const variantes: Record<string, "default" | "secondary" | "destructive" | "outline"> = {
    admin: "default",
    docente: "secondary",
    estudiante: "outline",
  };
  return (
    <Badge variant={variantes[rol] || "outline"}>
      {rol.charAt(0).toUpperCase() + rol.slice(1)}
    </Badge>
  );
};

return (
  <div className="space-y-4">
    {/* ===== FILTROS ===== */}
    <div className="flex items-center gap-4">
      <Select value={filtroRol} onChange={setFiltroRol}>
        <SelectTrigger className="w-[180px]">
          <SelectValue placeholder="Filtrar por rol" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los roles</SelectItem>
          <SelectItem value="admin">Admin</SelectItem>
          <SelectItem value="docente">Docente</SelectItem>
          <SelectItem value="estudiante">Estudiante</SelectItem>
        </SelectContent>
      </Select>

      <Select value={filtroEstado} onChange={setFiltroEstado}>
        <SelectTrigger className="w-[180px]">
          <SelectValue placeholder="Filtrar por estado" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los estados</SelectItem>
          <SelectItem value="activo">Activos</SelectItem>
          <SelectItem value="inactivo">Inactivos</SelectItem>
        </SelectContent>
      </Select>

      <Button variant="outline" size="sm" onClick={cargarUsuarios}>
        <RefreshCw className="mr-2 h-4 w-4" />
        Refrescar
      </Button>

      <span className="text-sm text-muted-foreground ml-auto">
        {usuariosFiltrados.length} usuario(s)
      </span>
    </div>

    {/* ===== TABLA ===== */}
    <div className="rounded-md border">
```

```

<Table>
  <TableHeader>
    <TableRow>
      <TableHead>Nombre</TableHead>
      <TableHead>Email</TableHead>
      <TableHead>Rol</TableHead>
      <TableHead>Estado</TableHead>
      <TableHead>Registro</TableHead>
      <TableHead className="text-right">Acciones</TableHead>
    </TableRow>
  </TableHeader>
  <TableBody>
    {loading ? (
      <TableRow>
        <TableCell colSpan={6} className="text-center py-8">
          Cargando usuarios...
        </TableCell>
      </TableRow>
    ) : usuariosFiltrados.length === 0 ? (
      <TableRow>
        <TableCell colSpan={6} className="text-center py-8">
          No se encontraron usuarios
        </TableCell>
      </TableRow>
    ) : (
      usuariosFiltrados.map((u) => (
        <TableRow key={u.id}>
          <TableCell className="font-medium">
            {u.nombre} {u.apellido}
          </TableCell>
          <TableCell>{u.email}</TableCell>
          <TableCell>{badgeRol(u.rol)}</TableCell>
          <TableCell>
            <Badge variant={u.activo ? "default" : "destructive"}>
              {u.activo ? "Activo" : "Inactivo"}
            </Badge>
          </TableCell>
          <TableCell>
            {new Date(u.created_at).toLocaleDateString("es-B0")}
          </TableCell>
          <TableCell className="text-right">
            <div className="flex justify-end gap-2">

              {/* Botón cambiar rol */}
              <Select
                onChange={(v) => cambiarRol(u.id, v)}
                value={u.rol}
              >
                <SelectTrigger className="w-[130px] h-8">
                  <Shield className="mr-1 h-3 w-3" />
                  <SelectValue placeholder="Rol" />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="estudiante">Estudiante</SelectItem>
                  <SelectItem value="docente">Docente</SelectItem>
                  <SelectItem value="admin">Admin</SelectItem>
                </SelectContent>
              </Select>

              {/* Botón activar/desactivar con confirmación */}
              <AlertDialog>
                <AlertDialogTrigger asChild>
                  <Button
                    variant={u.activo ? "destructive" : "default"}

```

```

        size="sm"
      >
      {u.activo ? (
        <><UserX className="mr-1 h-3 w-3" /> Desactivar</>
      ) : (
        <><UserCheck className="mr-1 h-3 w-3" /> Activar</>
      )}
    </Button>
  </AlertDialogTrigger>
  <AlertDialogContent>
    <AlertDialogHeader>
      <AlertDialogTitle>
        {u.activo
          ? "¿Desactivar usuario?"
          : "¿Activar usuario?"}
      </AlertDialogTitle>
      <AlertDialogDescription>
        {u.activo
          ? `${u.nombre} ${u.apellido} no podrá iniciar sesión.`
          : `${u.nombre} ${u.apellido} podrá acceder nuevamente.`}
      </AlertDialogDescription>
    </AlertDialogHeader>
    <AlertDialogFooter>
      <AlertDialogCancel>Cancelar</AlertDialogCancel>
      <AlertDialogAction
        onClick={() => toggleActivo(u.id, u.activo)}
      >
        Confirmar
      </AlertDialogAction>
    </AlertDialogFooter>
  </AlertDialogContent>
</AlertDialog>

</div>
</TableCell>
</TableRow>
))
)}
</TableBody>
</Table>
</div>
</div>
);
}

```

⚠ Atención: Este archivo es largo. Asegúrate de que todas las llaves y paréntesis estén correctamente cerrados. Si VS Code muestra errores de sintaxis, revisa que no falte ningún cierre.

4.3 — Anatomía del componente

Repasemos los patrones importantes usados en este componente:

Patrón	Componente shadcn/ui	Propósito
Filtros controlados	Select + SelectItem	El usuario filtra por rol y estado sin recargar la página
Badges semánticos	Badge con variantes	Indicadores visuales: default (admin/activo), destructive (inactivo), etc.
Diálogo de confirmación	AlertDialog	Pide confirmación antes de activar/desactivar
Select inline	Select dentro de TableCell	Cambio de rol directo desde la fila de la tabla
Refresh manual	Button con RefreshCw	Permite al admin recargar la lista manualmente
Filtrado en cliente	Array.filter()	Rápido para listas pequeñas (< 1000 registros)

Paso 5 — Página Admin: Usuarios

Ahora reemplazaremos la página placeholder de admin/usuarios con la página real que importa nuestro componente UsuariosTable.

5.1 — Actualizar la página

Reemplaza completamente el contenido de app/(dashboard)/admin/usuarios/page.tsx:

```
app/(dashboard)/admin/usuarios/page.tsx
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@components/ui/card";
import { Users } from "lucide-react";
import UsuariosTable from "@components/dashboard/admin/usuarios-table";

export default function AdminUsuariosPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      {/* Encabezado */}
      <div className="flex items-center gap-3">
        <Users className="h-8 w-8 text-primary" />
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            Gestión de Usuarios
          </h1>
          <p className="text-muted-foreground">
            Administra los usuarios del sistema, cambia roles y activa o
            desactiva cuentas.
          </p>
        </div>
      </div>

      {/* Card con la tabla */}
      <Card>
        <CardHeader>
          <CardTitle>Usuarios registrados</CardTitle>
          <CardDescription>
            Lista completa de todos los usuarios del sistema
          </CardDescription>
        </CardHeader>
        <CardContent>
          <UsuariosTable />
        </CardContent>
      </Card>
    </div>
  );
}
```

5.2 — Análisis del patrón

Observa que esta página es un Server Component (no tiene "use client"). Esto es intencional:

Componente	Tipo	Responsabilidad
page.tsx	Server Component	Estructura de la página, encabezado, Card
UsuariosTable	Client Component	Fetch de datos, filtros, interactividad

☑ **Patrón composición:** Este es el mismo patrón que usamos en la Entrega 4A con PerfilForm: la página (server) define la estructura visual, y el componente interactivo (client) maneja el estado y los datos.

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
components/ui/table.tsx	Nuevo (shadcn)	Componente Table auto-generado
components/ui/badge.tsx	Nuevo (shadcn)	Componente Badge auto-generado
components/ui/alert-dialog.tsx	Nuevo (shadcn)	Componente AlertDialog auto-generado
components/ui/select.tsx	Nuevo (shadcn)	Componente Select auto-generado
app/api/admin/usuarios/route.ts	Nuevo	GET: listar todos los usuarios (admin only)
app/api/admin/usuarios/[id]/route.ts	Nuevo	PUT: cambiar rol o activar/desactivar
components/dashboard/admin/usuarios-table.tsx	Nuevo	Tabla interactiva con filtros y acciones
app/(dashboard)/admin/usuarios/page.tsx	Modificado	Reemplaza placeholder con UsuariosTable

Verificación Final

Ejecuta `npm run dev` y verifica cada uno de estos puntos:

A. Acceso y protección

- ✓ Iniciar sesión con un usuario admin
- ✓ Navegar a `/admin/usuarios` desde el sidebar
- ✓ Si intentas acceder como estudiante o docente, el proxy te redirige

B. Tabla de usuarios

- ✓ La tabla muestra nombre, email, rol, estado y fecha de registro
- ✓ Los badges de rol muestran colores diferentes según el rol
- ✓ Los badges de estado muestran Activo (verde/default) o Inactivo (rojo/destructive)
- ✓ El contador muestra el número correcto de usuarios filtrados

C. Filtros

- ✓ Filtrar por rol: seleccionar Admin muestra solo admins
- ✓ Filtrar por estado: seleccionar Inactivos muestra solo desactivados
- ✓ Combinar filtros funciona correctamente (ej: Docente + Activo)
- ✓ El botón Refrescar recarga la lista desde el servidor

D. Acciones

- ✓ Cambiar rol con el Select inline actualiza el badge inmediatamente
- ✓ Hacer clic en Desactivar muestra el diálogo de confirmación
- ✓ Cancelar en el diálogo no hace ningún cambio
- ✓ Confirmar en el diálogo cambia el estado y muestra toast
- ✓ El admin NO puede modificarse a sí mismo (muestra error en toast)

E. Errores esperados

- ✓ Desloguear y llamar a `/api/admin/usuarios` devuelve 401

✓ Llamar como estudiante devuelve 403

☒ **Consejo:** Abre las DevTools del navegador (F12 → Network) para ver las llamadas fetch y sus respuestas. Esto te ayudará a diagnosticar errores.

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. Búsqueda por texto: Agrega un Input de búsqueda que filtre por nombre, apellido o email. Usa el componente Input de shadcn/ui que ya tienes instalado. El filtro debe funcionar en combinación con los filtros de rol y estado.
2. Ordenamiento de columnas: Haz que al hacer clic en el encabezado de una columna (ej: Nombre), la tabla se ordene alfabéticamente. Un segundo clic invierte el orden. Agrega un icono ChevronUp/ChevronDown de lucide-react al encabezado activo.
3. Contador por rol: Debajo de los filtros, agrega 3 StatsCard (el componente de la Entrega 4A) que muestren: Total de estudiantes, Total de docentes y Total de admins. Los números deben actualizarse al refrescar.
4. Confirmación para cambio de rol: Actualmente, el cambio de rol con el Select se ejecuta inmediatamente. Agrega un AlertDialog que pida confirmación antes de cambiar el rol, similar al que ya tienes para activar/desactivar.

i Para entregar: Implementa al menos 2 de los 4 ejercicios. Muestra capturas de pantalla en clase.

Próximo: Entrega 5A

En la siguiente entrega comenzaremos con la gestión de materias:

- API Route GET /api/admin/materias para listar todas las materias
- API Route POST /api/admin/materias para crear una nueva materia
- Esquema Zod de validación para materias (código, nombre, créditos, semestre)
- Página admin/materias con tabla similar a la de usuarios
- Asignar docente a una materia usando Select con lista de docentes

☒ **Preparación:** Asegúrate de tener al menos un usuario con rol "docente" creado, ya que lo necesitaremos para asignar docentes a materias.

Fin de la Entrega 4B